

UniHub: a vida universitária centralizada em um só lugar.

O UniHub é uma plataforma web para reunir eventos, fórum, moradias, agenda, notificações e perfis acadêmicos em um fluxo simples e organizado.

Flask + SQLAlchemy

Jinja2

Application Factory

Blueprints

1

Problema

O que observamos no contexto universitário

2

Solução

Como o UniHub centraliza a experiência

3

Técnico

Arquitetura, dados e qualidade

Integrantes: Matheus Pastore, Felipe Yuki e Davi Conde.



9

módulos de rotas

22

testes automatizados

3

perfis de acesso

O PROBLEMA

A vida universitária fica espalhada em vários canais.

As informações mais importantes aparecem em grupos, redes sociais e conversas separadas. Isso aumenta o ruído, faz o aluno perder contexto e dificulta a entrada de quem está chegando agora.

Grupos

Redes sociais

Conversas paralelas

EVENTOS

Divulgação dispersa

A informação aparece em vários lugares e perde visibilidade com facilidade.

MORADIA

Busca informal

O estudante depende de indicação, grupos e contatos que nem sempre são organizados.

FÓRUM

Dúvidas sem centralização

As respostas ficam soltas em conversas rápidas e perdem histórico.

CONSEQUÊNCIA

Mais tempo procurando, menos tempo participando

A rotina fica fragmentada e o estudante precisa gastar energia só para descobrir onde cada coisa está.

O impacto é prático.

Quem precisa de evento, moradia, dúvida acadêmica ou aviso oficial acaba montando o próprio mapa da universidade manualmente.

A SOLUÇÃO

Um hub para centralizar, organizar e conectar.

A proposta reúne as áreas mais importantes da rotina acadêmica em uma plataforma única, com navegação simples e papéis bem definidos.

Centralizar

Organizar

Conectar

Descobrir

eventos e avisos



Participar

fórum e agenda



Organizar

moradia e perfil



Acompanhar

notificações e progresso



Um só lugar para a experiência universitária.

Eventos, fórum, moradias, agenda, notificações e perfil convivem em uma mesma base.

EVENTOS

Agenda e descoberta

Divulgação, filtragem e salvamento de eventos ficam em um fluxo único.

FÓRUM

Dúvidas e colaboração

O espaço de discussão cria histórico, organização e apoio entre estudantes.

MORADIAS

Oferta e busca

Os anúncios aproximam quem oferece moradia e quem precisa se mudar.

PERFIL

Identidade acadêmica

O usuário ajusta seus dados, redes e preferências dentro da própria plataforma.

PARA QUEM É

O foco está no estudante, mas a organização atende três perfis de uso.

O UniHub foi pensado para quem precisa consumir conteúdo, publicar informações e manter a rotina acadêmica mais clara.

Proposta de valor

Centralizar informação reduz ruído, melhora a comunicação e deixa o uso da plataforma mais previsível para quem entra, representa e administra.

BENEFÍCIOS NA ROTINA

INTEGRAÇÃO

Menos dispersão

Um ambiente único para consultar o que importa.

ACESSO

Entrada mais clara

O caminho inicial da plataforma é simples e direto.

COMUNIDADE

Troca mais organizada

O conteúdo acadêmico fica mais fácil de acompanhar e participar.

PERFIS DA PLATAFORMA

ALUNO

Uso cotidiano

Eventos, moradias, fórum e perfil ficam disponíveis em um só ambiente.

REPRESENTANTE

Aluno verificado

Tem selo de verificação e pode criar eventos para movimentar a comunidade.

ADMIN

Controle do sistema

Publicação, moderação e permissões ficam concentradas no painel administrativo.

CASO DE USO DO ALUNO

O aluno entra, explora e usa a plataforma no mesmo fluxo.

Ele acessa a área pública, completa o perfil e segue para as áreas centrais sem perder o contexto da navegação.

A ideia é reduzir atrito.

Na prática, o aluno chega aos eventos, ao fórum, às moradias, à agenda e às notificações em um caminho contínuo. Quando ele também é representante, o perfil ganha selo de verificação e a opção de criar eventos.

O QUE O ALUNO ACESSA E FAZ

ACESSO

Entrada rápida

Login, cadastro e perfil inicial levam o aluno para o ambiente principal sem etapas extras.

CONTEÚDO

Áreas centrais

Eventos, fórum, moradias e agenda ficam reunidos no mesmo espaço, com navegação curta.

INTERAÇÃO

Rotina contínua

Salvar, responder e acompanhar notificações mantém o aluno conectado ao que importa.

PASSO A PASSO DO ALUNO

1

Entrar

login ou cadastro

2

Completar

perfil e dados

3

Explorar

eventos, fórum e moradias

4

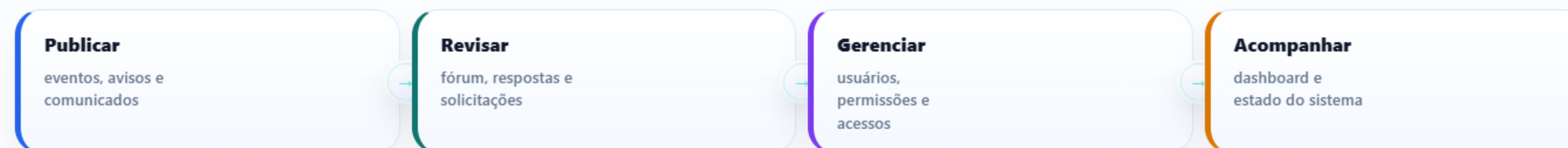
Interagir

salvar, responder e acompanhar

CASO DE USO DO ADMINISTRADOR

Quem administra a plataforma segue um fluxo próprio.

Administradores publicam conteúdo, revisam interações e controlam permissões em um espaço separado da experiência do aluno.



Papéis separados, responsabilidades claras.

O painel administrativo aparece só para quem tem permissão, sem misturar os controles internos com o que o estudante vê.

O QUE O ADMINISTRADOR ACESSA E FAZ

ACESSO

Área restrita

Só perfis autorizados entram nas telas administrativas e nos controles internos.

OPERAÇÃO

Publicação e revisão

Conteúdo, tópicos e respostas passam por acompanhamento antes de irem para a comunidade.

CONTROLE

Visão administrativa

Usuários, permissões e estado do sistema ficam centralizados em um só painel.

O QUE FOI ENTREGUE

O projeto cobre as áreas centrais da experiência.

A aplicação final não entrega só uma interface. Ela organiza entrada, participação, publicação, persistência e administração em módulos separados.

LANDING

Porta de entrada pública

Apresenta o projeto e direciona o usuário para login ou cadastro.

AUTENTICAÇÃO

Login, cadastro e sessão

Fluxos com proteção de área autenticada e dados validados no servidor.

DASHBOARD

Painel principal

Reúne atividades recentes, avisos, conexões e atalhos úteis.

PERFIL

Dados e configurações

O usuário edita bio, curso, contatos e preferências pessoais.

FÓRUM

Tópicos e respostas

O espaço organiza dúvidas, materiais, avisos e colaboração.

EVENTOS

Agenda e participação

Os eventos podem ser criados, filtrados, salvos e acompanhados.

MORADIAS

Anúncios e busca

Os anúncios ajudam a encontrar e divulgar moradia com filtros claros.

ADMIN

Moderação e controle

Usuários, conteúdo e status são tratados de forma separada e segura.

A estrutura cresceu sem perder organização.

O repositório foi dividido em módulos pequenos, com responsabilidades claras e uma base de testes que permite evoluir o sistema sem retrabalho.

O tamanho não virou bagunça.

A separação entre camadas mantém o código previsível, mesmo com várias áreas funcionando ao mesmo tempo.

1

fábrica de aplicação

9

módulos de rotas

7

formulários principais

7

entidades do domínio

22

testes automatizados

61

templates Jinja

3 perfis de acesso

1 banco principal

múltiplas áreas de uso

A aplicação nasce no `app.py` e se divide em módulos pequenos.

O `create_app()` lê a configuração, inicializa as extensões e registra as blueprints por área. Esse fluxo deixa o projeto previsível, fácil de testar e simples de ampliar.

FLUXO DE INICIALIZAÇÃO

- 1** `.env` + `FLASK_CONFIG`
Define o ambiente e a configuração antes da montagem.
- 2** `app.py` + `create_app()`
Centraliza a criação da aplicação e a ordem de inicialização.
- 3** `config` + `ext`
Carrega banco, login, CSRF e Swagger em módulos separados.
- 4** `views` + `seed` + `tests`
Registra blueprints e reaproveita a mesma base nos testes.

1

`app.py`

Ponto de entrada da fábrica da aplicação.

2

`unihub/config.py`

Escolha de configuração por ambiente.

3

`unihub/ext/`

Banco, login, CSRF e Swagger inicializados separadamente.

4

`unihub/views/`

Rotas organizadas por responsabilidade, sem arquivo único gigante.

5

`seed.py` + `tests/base.py`

Os dados iniciais e os testes usam a mesma fábrica, o que mantém a execução previsível.

Por que isso ajuda

Como a aplicação é montada por etapas, os testes criam o app com `TESTING=True` e memória sem duplicar a inicialização.

O repositório separa aplicacao, telas, testes e apoio por responsabilidade.

A raiz concentra a fabrica da aplicacao. A partir dela, views, forms, models, services, templates, static, docs e tests ficam em blocos claros.

ESTRUTURA DE PASTAS

Base real do repositório original, sem a pasta da apresentação

UniHubTemp/

app.py	<- create_app() / entrada da aplicacao
unihub/	<- pacote principal
views/	<- blueprints e rotas
models/	<- ORM e entidades
forms/	<- formulários e validacao
services/	<- logica de negocio
ext/	<- extensoes Flask
utils/	<- helpers e utilitários
templates/	<- telas Jinja2
static/	<- CSS + imagens
tests/	<- testes automatizados
migrations/	<- schema e versoes
docs/	<- documentacao e imagens

Camadas: **Views** -> **Services** -> **Models** -> **Banco**

Visao geral da base de codigo antes da camada de dados.

Os dados seguem um modelo relacional simples e legível.

O banco liga usuários, eventos, fórum, moradias, notificações e agenda. O projeto usa SQLAlchemy e migrations para evoluir o schema com segurança.

```
Usuario 1 -> Evento
Usuario 1 -> ForumTopico
Usuario 1 -> ForumResposta
Usuario 1 -> Moradia
Usuario 1 -> Notificacao
Usuario 1 -> AgendaEvento
Evento 1 -> AgendaEvento
ForumTopico 1 -> ForumResposta
```

USUARIOS

Perfil, papel e contato

O modelo guarda curso, período, cidade, redes e tipo de acesso.

EVENTOS

Organização e agenda

Quem cria também organiza, e o usuário pode salvar eventos na própria agenda.

FÓRUM

Tópicos e respostas

A discussão fica estruturada com status, autor e histórico de atualização.

MORADIAS

Anúncios com regras claras

Status, preço, vagas e contatos são persistidos com validação no banco.

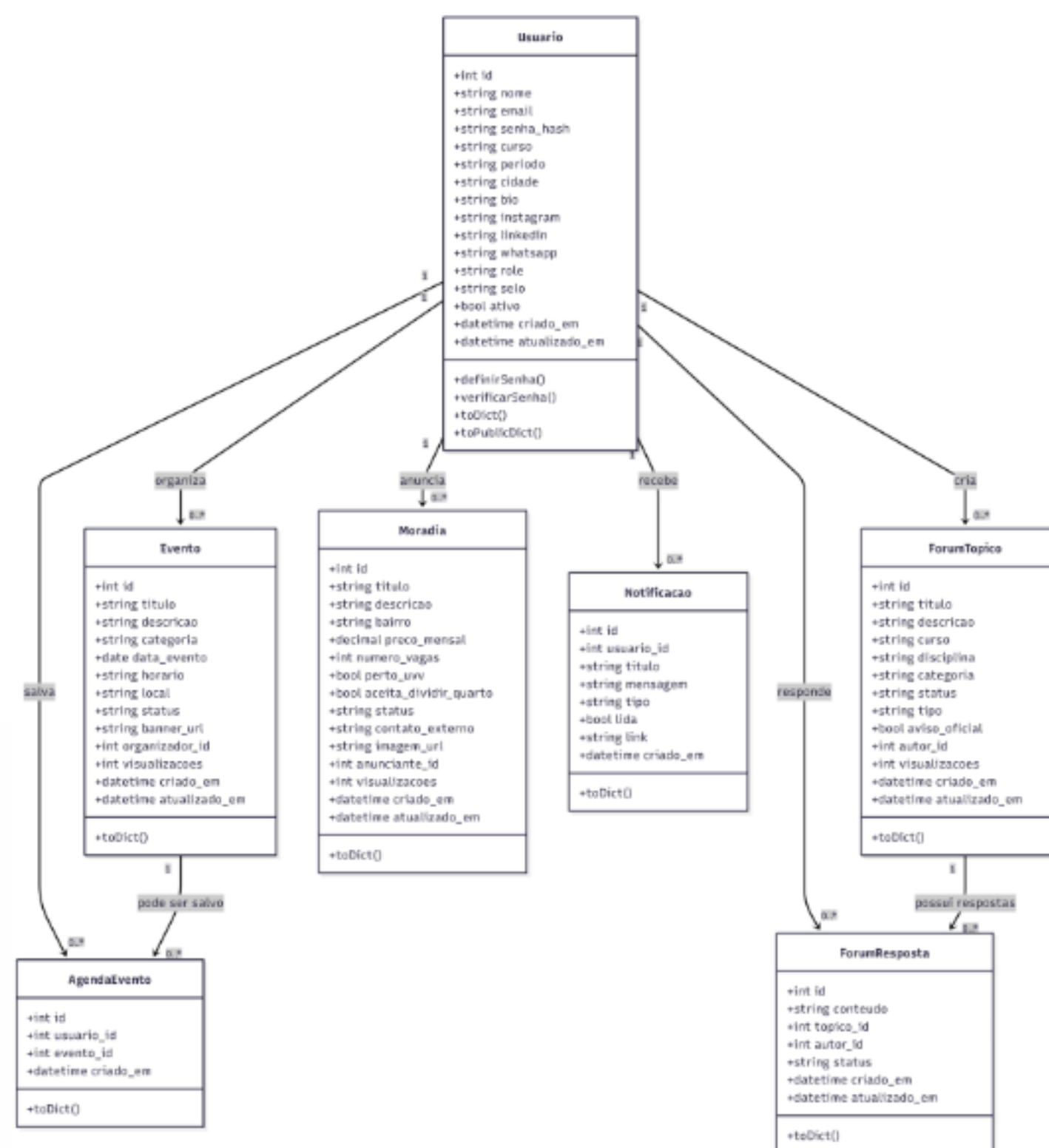
NOTIFICAÇÕES

Estado de leitura e vínculo com o usuário

Cada notificação pertence a uma conta específica, com suporte a leitura, filtro e link de origem.

A estrutura relacional do UniHub em um único mapa.

O diagrama mostra como usuários, eventos, fórum, moradias, agenda e notificações se conectam para sustentar o sistema.



Formulários, interface e qualidade caminham juntos.

O servidor valida os dados, os templates organizam a experiência e os testes protegem os fluxos principais.

`LoginForm` e `CadastroForm` validação de acesso, senha e dados iniciais do usuário

`PerfilForm` edição de dados pessoais, bio e redes sociais

`TopicoForm` e `RespostaForm` conteúdo do fórum com regras no servidor

`EventoForm` e `MoradiaForm` publicação de eventos e anúncios com validação clara

O lado visual e a organização técnica

INTERFACE

Jinja2 + layouts compartilhados

As páginas públicas e autenticadas reaproveitam componentes e mantêm coerência visual.

RESPONSIVIDADE

Interface pronta para tela pequena

Os blocos se reorganizam e continuam legíveis em diferentes tamanhos de tela.

QUALIDADE

22 testes e ambiente isolado

Os testes usam `unittest`, SQLite em memória e a mesma fábrica da aplicação.

BOAS PRÁTICAS

Reprodutibilidade

`.env.example`, migrations, seed e separação por pastas deixam o projeto mais claro.

FORMULÁRIOS

O servidor valida antes de gravar qualquer dado relevante.

INTERFACE

Jinja2 e layouts compartilhados mantêm a navegação consistente.

QUALIDADE

Testes automatizados protegem os fluxos principais da aplicação.

UniHub pronto para apresentar e evoluir.

O projeto junta problema real, solução simples e uma base modular que pode crescer com novos módulos, telas e regras sem perder organização.

[Voltar ao início](#)[Rever estrutura](#)[Flask](#)[SQLAlchemy](#)[Templates](#)[Testes](#)**7**

entidades principais

22

testes automatizados

61

templates Jinja